Java Backend Interview Series

Sub-Chapter 1.11

Multithreading Fundamentals

Thread Lifecycle . Synchronization . JMM . Virtual Threads

Motivation + Before/After Code + Import Blocks + 30 Q&A + Pitfalls + Tips

1 Thread Creation and Lifecycle

1.1 Introduction

Java threads are units of concurrent execution managed by the JVM and scheduled by the OS. A thread progresses through well-defined states. The preferred approach is submitting Runnable/Callable to an ExecutorService rather than managing Thread objects directly.

1.2 Thread Creation -- Three Approaches

```
import java.util.concurrent.Callable;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.Future;
// 1. Extending Thread (discouraged -- couples task with execution mechanism)
class MyThread extends Thread {
public void run() { System.out.println("Thread: " + getName()); }
}
new MyThread().start();

// 2. Implementing Runnable (preferred for fire-and-forget tasks)
Runnable task = () -> System.out.println("Runnable on " +
Thread.currentThread().getName());
Thread t = new Thread(task, "worker-1");
t.start(); // never call run() directly -- runs on current thread!

// 3. Callable + Future (tasks that return values or throw checked exceptions)
Callable<Integer> compute = () -> {
Thread.sleep(100);
return 42;
};
ExecutorService exec = Executors.newFixedThreadPool(4);
Future<Integer> future = exec.submit(compute);
Integer result = future.get(); // blocks until result ready
exec.shutdown();
```

Thread creation: extend Thread, implement Runnable, use Callable

1.3 Thread Lifecycle States

State	Meaning	How to Enter
NEW	Created but start() not called	new Thread(...)
RUNNABLE	Running or ready (OS-scheduled)	t.start()

State	Meaning	How to Enter
BLOCKED	Waiting to acquire a monitor lock	Contended synchronized block
WAITING	Indefinitely waiting for notification	Object.wait(), LockSupport.park()
TIMED_WAITING	Waiting with a timeout	Thread.sleep(), wait(ms), join(ms)
TERMINATED	run() has completed	Normal return or uncaught exception

1.4 Daemon Threads

```
// Daemon threads: JVM exits when only daemon threads remain

// (background housekeeping -- GC, monitoring, timers)

Thread daemon = new Thread(() -> {

while (true) {

System.out.println("heartbeat");

Thread.sleep(1000);

}

});

daemon.setDaemon(true); // MUST set before start()

daemon.start();

// When the last non-daemon thread finishes, the JVM kills this thread


// Check state

Thread t = Thread.currentThread();

t.getState(); // Thread.State enum

t.getId(); // unique long ID

t.getName();

t.getPriority(); // 1 (MIN) to 10 (MAX), default 5 (NORM)
```

Daemon threads and thread inspection

WARNING: Never call run() directly -- it executes the Runnable on the calling thread, not a new thread. Always call start(). Calling start() twice on the same Thread throws IllegalThreadStateException.

2 Synchronization

2.1 Introduction

Without synchronization, concurrent access to shared mutable state produces data races -- unpredictable results caused by interleaved read-modify-write sequences. Java synchronization uses intrinsic locks (monitors) to enforce mutual exclusion and establish happens-before relationships.

2.2 synchronized Method and Block

```
// synchronized METHOD -- lock is the instance (this)

class Counter {

private int count = 0;


// Entire method is atomic -- only one thread at a time

public synchronized void increment() { count++; }

public synchronized int get() { return count; }


// STATIC synchronized -- lock is the Class object

public static synchronized void staticIncrement() { staticCount++; }

}


// synchronized BLOCK -- fine-grained: lock only what you must

class BetterCounter {

private final Object lock = new Object(); // private dedicated lock

private int count = 0;


public void increment() {

synchronized (lock) { // only this block is locked

count++;

}

// expensiveOperation() runs without holding the lock

expensiveOperation();

}

}


// Intrinsic lock is REENTRANT: a thread that holds the lock can re-enter
```

```
class Reentrant {

synchronized void methodA() {

methodB(); // OK -- same thread already holds the lock

}

synchronized void methodB() { System.out.println("B"); }

}
```

synchronized method, synchronized block, reentrant locking

2.3 Static vs Instance Synchronization

```
// Instance lock (this) -- protects instance state, one lock per object

public synchronized void instanceMethod() { ... }


// Static lock (Counter.class) -- protects static state, one lock per class

public static synchronized void staticMethod() { ... }


// These two locks are INDEPENDENT:

// Thread A can hold instance lock while Thread B holds class lock simultaneously


// WRONG: mixing static and instance state under instance lock

class Broken {

private static int sharedCount = 0;

public synchronized void increment() {

sharedCount++; // BUG: different objects have different locks!

}

}

// FIX: use static synchronized or a static lock object

class Fixed {

private static int sharedCount = 0;

private static final Object LOCK = new Object();

public void increment() {

synchronized (LOCK) { sharedCount++; }

}

}
```

Static vs instance synchronization -- common mistake

WARNING: Using synchronized on public fields (synchronized(this)) exposes the lock to external code that can deadlock your class by acquiring the same lock. Always use a private final Object as the lock.

3 The volatile Keyword

3.1 Introduction

volatile guarantees visibility of writes across threads via happens-before, but provides NO atomicity for compound operations like check-then-act or read-modify-write. Use it for flags, not counters.

3.2 Visibility Problem Without volatile

```
// WITHOUT volatile: compiler/CPU may cache the flag in a register

// Thread A may never see the change made by Thread B

class StopTask implements Runnable {

private boolean stop = false; // NOT volatile -- dangerous!


public void run() {

while (!stop) { // Thread A may see stale false forever

doWork();

}

}

public void requestStop() { stop = true; } // Thread B writes

}


// WITH volatile: write to stop is immediately visible to all threads

class SafeStopTask implements Runnable {

private volatile boolean stop = false; // volatile visibility guarantee


public void run() {

while (!stop) { doWork(); } // always reads fresh value

}

public void requestStop() { stop = true; }

}
```

volatile flag: visibility guarantee

3.3 What volatile Does NOT Guarantee

```
import java.util.concurrent.atomic.AtomicInteger;
```

```
// volatile does NOT make compound operations atomic

class BrokenCounter {

private volatile int count = 0;
```

```
// NOT thread-safe! count++ is 3 ops: read, increment, write
public void increment() { count++; } // race condition!
}

// Use AtomicInteger for atomic compound operations
class SafeCounter {
private final AtomicInteger count = new AtomicInteger(0);
public void increment() { count.incrementAndGet(); } // atomic
public int get() { return count.get(); }
}

// volatile is ENOUGH when:
// 1. Single writer, multiple readers (flag pattern)
// 2. Writes are independent (not based on previous value)
// volatile is NOT ENOUGH when:
// 1. Read-modify-write (count++) -- need AtomicXxx
// 2. Check-then-act (if(ready) use(data)) -- need synchronized
```

volatile does not provide atomicity

WARNING: A common interview trap: volatile int count; count++ is NOT thread-safe even though count is volatile. Use AtomicInteger.incrementAndGet() or synchronized methods for read-modify-write operations.

TIP: Use volatile for: (1) stop/shutdown flags, (2) double-checked locking (Java 5+), (3) publishing a reference to an immutable object. Use AtomicXxx or synchronized for anything requiring compound atomicity.

4 wait / notify / notifyAll

4.1 Introduction

wait() and notify() enable threads to communicate about shared state. A thread calls wait() to release the monitor and pause; another calls notify() to wake a waiting thread. Both must be called while holding the monitor lock.

4.2 Producer-Consumer with wait / notify

```
import java.util.Queue;
import java.util.LinkedList;
class BoundedBuffer<T> {
private final Queue<T> queue = new LinkedList<>();
private final int capacity;

BoundedBuffer(int capacity) { this.capacity = capacity; }

public synchronized void put(T item) throws InterruptedException {
// WHILE not IF -- guard against spurious wakeups
while (queue.size() == capacity) {
wait(); // releases lock; waits until notified
}
queue.add(item);
notifyAll(); // wake all waiting threads (prefer over notify())
}

public synchronized T take() throws InterruptedException {
while (queue.isEmpty()) {
wait(); // releases lock; waits until notified
}
T item = queue.poll();
notifyAll();
return item;
}
}
```

Producer-consumer with wait/notifyAll (while loop is mandatory)

4.3 Spurious Wakeups and Why We Use while

```
// Spurious wakeup: a thread may wake from wait() without notify() being called
// This is legal per the JVM spec (POSIX pthreads behavior)
```

```
// WRONG -- using if: thread re-enters after spurious wakeup without re-checking

synchronized (lock) {

if (!condition) { // BUG: checked only once

lock.wait();

}

useSharedData(); // might run even though condition is still false!

}


// CORRECT -- using while: always re-checks condition after wakeup

synchronized (lock) {

while (!condition) { // re-check on every wakeup

lock.wait();

}

useSharedData(); // condition is guaranteed true here

}
```

Spurious wakeup: always use while, never if

WARNING: Always call wait() inside a while loop, never an if. Spurious wakeups (thread wakes without notify) are legal. Using if instead of while allows threads to proceed when the condition is still false.

TIP: Prefer notifyAll() over notify(). notify() wakes only one waiting thread (non-deterministically). notifyAll() wakes all waiting threads -- each re-checks its while condition, ensuring correctness at the cost of slightly more CPU.

5 Thread Interruption and Cooperative Cancellation

5.1 Introduction

Java uses cooperative cancellation: one thread requests another to stop via interrupt(), and the target thread is expected to check and honour the request. Forcing termination (Thread.stop()) was deprecated because it leaves shared state inconsistent.

5.2 interrupt(), isInterrupted(), InterruptedException

```
// interrupt() sets the interrupt flag on the target thread

Thread worker = new Thread(() -> {

try {

while (!Thread.currentThread().isInterrupted()) {

doWork();

}

System.out.println("Worker cleanly stopped");

} catch (InterruptedException e) {

// Blocking ops (sleep, wait, join) throw InterruptedException

// when the interrupt flag is set

Thread.currentThread().interrupt(); // restore the flag!

System.out.println("Interrupted during blocking op");

}

}, "worker");


worker.start();

Thread.sleep(100);

worker.interrupt(); // request cancellation


// Thread.interrupted() -- clears the flag (side effect!)

// Thread.currentThread().isInterrupted() -- checks without clearing
```

Cooperative cancellation via interrupt()

5.3 Interrupt Policy -- What to Do with InterruptedException

```
// Option 1: Propagate (best -- let the caller decide)

void doTask() throws InterruptedException {

Thread.sleep(1000); // propagates InterruptedException
```

```
}


// Option 2: Restore flag then return/exit

void doTaskSafe() {

try {

Thread.sleep(1000);

} catch (InterruptedException e) {

Thread.currentThread().interrupt(); // MUST restore the flag

return; // or break out of loop

}

}


// WRONG: swallowing InterruptedException without restoring the flag

void broken() {

try {

Thread.sleep(1000);

} catch (InterruptedException e) {

// BUG: flag cleared by catch, caller cannot detect interruption

}

}
```

InterruptedException: propagate or restore -- never swallow

WARNING: Never swallow InterruptedException silently (empty catch block). Either propagate it (throws InterruptedException) or restore the flag with Thread.currentThread().interrupt() before returning. Swallowing it hides cancellation signals from callers.

6 Thread Pools and ThreadPoolExecutor

6.1 Introduction

Creating a new Thread for every task is expensive (OS-level). Thread pools reuse threads across tasks, bound the number of concurrent threads, and provide queue-based task management. ThreadPoolExecutor is the central implementation; Executors provides convenient factory methods.

6.2 ThreadPoolExecutor -- Full Constructor

```
import java.util.concurrent.ThreadPoolExecutor;
import java.util.concurrent.TimeUnit;
import java.util.concurrent.ArrayBlockingQueue;
import java.util.concurrent.Future;
import java.util.concurrent.atomic.AtomicInteger;
ThreadPoolExecutor pool = new ThreadPoolExecutor(
4, // corePoolSize -- always-alive threads
16, // maximumPoolSize -- max threads under load
60L, TimeUnit.SECONDS, // keepAliveTime -- idle non-core threads die
new ArrayBlockingQueue<>(500), // bounded work queue (important!)
r -> { // ThreadFactory -- custom names
Thread t = new Thread(r, "worker-" + threadNum.incrementAndGet());
t.setDaemon(false);
return t;
},
new ThreadPoolExecutor.CallerRunsPolicy() // rejection policy
);

// Lifecycle
pool.shutdown(); // stop accepting new tasks, finish queued tasks
pool.shutdownNow(); // interrupt running tasks, drain queue
boolean done = pool.awaitTermination(30, TimeUnit.SECONDS);

// submit vs execute
pool.execute(runnable); // fire-and-forget
Future<String> f = pool.submit(callable); // returns Future
```

ThreadPoolExecutor full constructor and lifecycle

6.3 Executors Factory Methods and Their Dangers

Factory Method	Config	Danger
newFixedThreadPool(n)	core=max=n, unbounded LinkedBlockingQueue	Unbounded queue causes OOM under sustained load
newCachedThreadPool()	core=0, max=Integer.MAX_VALUE, SynchronousQueue	Unbounded threads -- can create millions under burst
newSingleThreadExecutor()	core=max=1, unbounded queue	Same OOM risk as fixed pool; serializes all tasks
newScheduledThreadPool(n)	core=n, unbounded DelayedWorkQueue	Unbounded queue; missed tasks can pile up
newVirtualThreadPerTaskExecutor()	One virtual thread per task (Java 21)	ThreadLocals leak; avoid synchronized in tasks

WARNING: Executors.newFixedThreadPool() uses an UNBOUNDED LinkedBlockingQueue. Under sustained overload, millions of tasks queue up, exhausting heap memory. Always use ThreadPoolExecutor with a bounded queue in production code.

TIP: Rejection policies: AbortPolicy (throws RejectedExecutionException -- default), CallerRunsPolicy (caller runs the task -- provides back-pressure), DiscardPolicy (silently drops), DiscardOldestPolicy (drops oldest queued task).

7 Deadlock -- Conditions, Detection, Prevention

7.1 Introduction

A deadlock occurs when two or more threads wait for each other to release locks they hold, forming a circular dependency. The four necessary conditions (Coffman conditions) must ALL hold for deadlock to occur.

7.2 Deadlock Demo

```
// Classic deadlock: two threads, two locks, opposite acquisition order

Object lockA = new Object();

Object lockB = new Object();


Thread t1 = new Thread(() -> {

synchronized (lockA) { // T1 acquires A

Thread.sleep(50);

synchronized (lockB) { // T1 waits for B (held by T2)

System.out.println("T1 done");

}

}

});


Thread t2 = new Thread(() -> {

synchronized (lockB) { // T2 acquires B

Thread.sleep(50);

synchronized (lockA) { // T2 waits for A (held by T1) -- DEADLOCK

System.out.println("T2 done");

}

}

});

t1.start(); t2.start(); // both threads block forever
```

Classic deadlock: two locks in opposite order

7.3 Prevention -- Lock Ordering and tryLock

```
import java.util.concurrent.locks.ReentrantLock;

import java.util.concurrent.TimeUnit;

// Fix 1: Consistent lock ordering -- always acquire A before B
```

```
Thread fixed1 = new Thread(() -> {
synchronized (lockA) {
synchronized (lockB) { // same order in both threads
System.out.println("T1 done -- no deadlock");
}
}
});

// Fix 2: tryLock with timeout (ReentrantLock)
ReentrantLock lockX = new ReentrantLock();
ReentrantLock lockY = new ReentrantLock();

void transferWithTryLock() throws InterruptedException {
while (true) {
if (lockX.tryLock(50, TimeUnit.MILLISECONDS)) {
try {
if (lockY.tryLock(50, TimeUnit.MILLISECONDS)) {
try {
doTransfer();
return; // success
} finally { lockY.unlock(); }
}
} finally { lockX.unlock(); }
}
Thread.sleep(1); // backoff before retry
}
}
```

Deadlock prevention: lock ordering and tryLock timeout

Coffman Condition	Meaning	Prevention
Mutual Exclusion	Resources held exclusively	Use read/write locks; reduce exclusivity
Hold and Wait	Hold one lock while waiting for another	Acquire all locks at once; tryLock
No Preemption	Locks cannot be forcibly taken	Use tryLock with timeout (forcible retry)
Circular Wait	Circular chain of wait dependencies	Enforce global lock ordering

WARNING: Detect deadlocks in production with: jstack <pid> or VisualVM or JMX ThreadMXBean.findDeadlockedThreads(). Enable -XX:+PrintConcurrentLocks with jstack for lock ownership details.

8 Race Conditions and Atomic Variables

8.1 Introduction

A race condition occurs when the outcome depends on the interleaving of operations across threads. Two classic patterns: check-then-act (read a condition, then act based on it) and read-modify-write (read, compute, write back). Both must be atomic.

8.2 Check-Then-Act and Read-Modify-Write

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.ConcurrentHashMap;
// Check-then-act race condition
if (!map.containsKey(key)) { // T1 checks: not present
map.put(key, computeValue()); // T2 also checks: not present
} // Both insert -- duplicate work!
// Fix: ConcurrentHashMap.computeIfAbsent (atomic)
map.computeIfAbsent(key, k -> computeValue());

// Read-modify-write race condition
int count = 0; // shared field
// Thread A: reads count=0, Thread B: reads count=0
// Thread A: writes count=1, Thread B: writes count=1 -- lost update!
count++; // NOT atomic without synchronization

// Fix 1: synchronized
synchronized (this) { count++; }

// Fix 2: AtomicInteger (lock-free, faster)
AtomicInteger atomicCount = new AtomicInteger(0);
atomicCount.incrementAndGet(); // atomic CAS loop
atomicCount.compareAndSet(expected, newVal); // conditional update
```

Race conditions: check-then-act, read-modify-write

8.3 AtomicInteger Internals

```
import java.util.concurrent.atomic.AtomicInteger;
import java.util.concurrent.atomic.AtomicReference;
import java.util.concurrent.atomic.LongAdder;
AtomicInteger counter = new AtomicInteger(0);

// Common operations
int val = counter.get();
```

```
int old = counter.getAndIncrement(); // returns old value, increments

int now = counter.incrementAndGet(); // increments, returns new value

int prev = counter.getAndAdd(5); // adds 5, returns old value

// compareAndSet (CAS) -- heart of all atomic operations

boolean swapped = counter.compareAndSet(5, 10);

// Atomically: if current==5, set to 10 and return true; else return false

// AtomicReference -- for object references

AtomicReference<String> ref = new AtomicReference<>("initial");

ref.compareAndSet("initial", "updated"); // atomic pointer swap

// LongAdder -- faster than AtomicLong under high contention

// (distributes increments across cells, merges on read)

LongAdder adder = new LongAdder();

adder.increment();

long total = adder.sum(); // sum of all cells
```

AtomicInteger CAS, AtomicReference, LongAdder

TIP: Use LongAdder instead of AtomicLong when you have many threads incrementing a counter and rarely reading the total (e.g., metrics collection). LongAdder reduces contention by keeping per-stripe counters.

9 Java Memory Model (JMM)

9.1 Introduction

The JMM defines which values a thread is guaranteed to see when it reads a variable written by another thread. Without JMM guarantees, compilers and CPUs may reorder instructions and cache values in registers -- producing counterintuitive results across threads. The JMM formalizes these rules through happens-before relationships.

9.2 Happens-Before Rules

Rule	Statement
Program Order	Each action in a thread happens-before any later action in that same thread
Monitor Lock	An unlock of a monitor happens-before every subsequent lock of that monitor
Volatile Write	A volatile write happens-before every subsequent read of that same variable
Thread Start	Thread.start() happens-before any action in the started thread
Thread Join	All actions in a thread happen-before Thread.join() returns in the joiner
Transitivity	If A happens-before B and B happens-before C, then A happens-before C

9.3 Visibility and Reordering Demo

```
// WITHOUT synchronization: reordering can cause wrong results

int x = 0, y = 0;

boolean flag = false;


// Thread A (writer)

void writer() {

x = 42; // (1)

flag = true; // (2) -- CPU may reorder (2) before (1) !

}


// Thread B (reader)

void reader() {

if (flag) { // (3) might see flag=true

System.out.println(x); // (4) but x might still be 0!

}

}


// FIX 1: make flag volatile
```

```
// volatile write to flag (2) happens-before volatile read of flag (3)

// therefore x=42 (1) is visible when (4) executes

volatile boolean safeFlag = false;


// FIX 2: synchronize both writer and reader on the same monitor

// monitor unlock in writer happens-before lock in reader
```

Visibility and reordering: the classic double-flag problem

9.4 Double-Checked Locking (Safe in Java 5+)

```
// Safe double-checked locking requires volatile (Java 5+)

class Singleton {

// volatile prevents reordering of object construction

private static volatile Singleton instance;


public static Singleton getInstance() {

if (instance == null) { // first check (no lock)

synchronized (Singleton.class) {

if (instance == null) { // second check (with lock)

instance = new Singleton();

// Without volatile: another thread might see a

// partially constructed object!

}

}

}

return instance;

}

// BETTER: use enum or holder class pattern (no double-checked locking needed)

}
```

Double-checked locking requires volatile

> WARNING: Double-checked locking WITHOUT volatile is broken in Java -- the JVM can reorder constructor writes relative to the reference assignment, causing a second thread to see a partially initialized object. Always use volatile for double-checked locking.

10 Virtual Threads -- Project Loom (Java 21)

10.1 Introduction

Platform threads map 1:1 to OS threads (~1 MB stack each). For I/O-bound services, the constraint is thread count, not CPU. Virtual threads (Java 21) are lightweight JVM-managed threads that unmount from their carrier (platform) thread during blocking I/O, freeing the carrier for other work. The JVM can run millions of virtual threads on a small carrier pool.

10.2 Creating and Using Virtual Threads

```
import java.time.Duration;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.stream.IntStream;
// Thread.ofVirtual() -- direct creation
Thread vt = Thread.ofVirtual().name("worker-1").start(() -> {
Thread.sleep(Duration.ofSeconds(1)); // yields carrier -- NOT blocked!
System.out.println("Done: " + Thread.currentThread());
// VirtualThread[#23,worker-1]/runnable@ForkJoinPool-1-worker-1
});

// Idiomatic: one virtual thread per task via ExecutorService
try (ExecutorService exec = Executors.newVirtualThreadPerTaskExecutor()) {
IntStream.range(0, 100_000).forEach(i ->
exec.submit(() -> {
Thread.sleep(Duration.ofSeconds(1)); // yields, not blocks
return fetchUser(i);
})
);
} // auto-closes: waits for all tasks

// Spring Boot 3.2+ -- one config line enables virtual threads:
// spring.threads.virtual.enabled: true
```

Virtual thread creation and executor usage

10.3 Platform vs Virtual Thread Comparison

Aspect	Platform Thread	Virtual Thread
Backed by	OS thread (1:1 mapping)	JVM continuation (M:N on carrier pool)
Stack size	~1 MB fixed	Few KB, grows dynamically
Max concurrent	Thousands	Millions

Aspect	Platform Thread	Virtual Thread
Thread.sleep()	Blocks OS thread	Yields carrier -- efficient
synchronized	Works fine	PINS carrier -- performance hit
I/O blocking	Blocks OS thread	Unmounts carrier -- other tasks run
ThreadLocal	Works; pooled -- leaks likely	Isolated per task; less risk in per-task model

10.4 Carrier Pinning -- The Main Pitfall

```
import java.util.concurrent.locks.ReentrantLock;
// synchronized PINS the virtual thread to its carrier
// -- the carrier cannot run other virtual threads while pinned
void badMethod() {
synchronized (this) {
jdbcOperation(); // I/O blocks carrier -- ruins scalability!
}
}

// FIX: replace synchronized with ReentrantLock
private final ReentrantLock lock = new ReentrantLock();
void goodMethod() throws InterruptedException {
lock.lock();
try {
jdbcOperation(); // carrier is freed during I/O -- OK
} finally {
lock.unlock();
}
}

// Diagnose pinning:
// -Djdk.tracePinnedThreads=full -- logs pinned threads to stdout
```

Carrier pinning: synchronized vs ReentrantLock

> WARNING: synchronized blocks PIN the virtual thread to its carrier, eliminating the scalability benefit. Replace synchronized with ReentrantLock for any critical section that performs I/O. Diagnose with: -Djdk.tracePinnedThreads=full

11 Interview Q and A -- Multithreading

30 questions covering standard and advanced interview depth. [ADV] marks senior-level questions.

Q: What are the six thread states in Java?

A: NEW (created, not started), RUNNABLE (running or ready), BLOCKED (waiting for monitor lock), WAITING (indefinitely waiting via wait/park/join), TIMED_WAITING (waiting with timeout via sleep/wait(ms)/join(ms)), TERMINATED (run() completed).

Q: What is the difference between Runnable and Callable?

A: Runnable.run() returns void and cannot throw checked exceptions. Callable<V>.call() returns a value V and can throw checked exceptions. Use Callable with ExecutorService.submit() to get a Future<V>.

Q: Why should you never call thread.run() directly?

A: run() executes the Runnable on the CURRENT thread, not a new thread. No concurrency occurs. Always call start() which creates a new thread, registers it with the JVM scheduler, and eventually calls run() on the new thread.

Q: What does synchronized guarantee?

A: 1) Mutual exclusion: only one thread at a time executes the synchronized block/method. 2) Visibility: when a thread exits a synchronized block, all writes made inside are visible to the next thread that enters a synchronized block on the same monitor.

Q: What is the difference between synchronized method and synchronized block?

A: A synchronized method locks the entire method using 'this' (or the class object for static). A synchronized block locks only the specified section and allows you to use a different, more granular lock object. Synchronized blocks are preferred for performance and flexibility.

Q: What does the volatile keyword guarantee?

A: Visibility: a volatile write is visible to all threads that subsequently read the same variable. Ordering: volatile write happens-before volatile read. No atomicity: count++ is not atomic even on a volatile int.

Q: When should you use volatile instead of synchronized?

A: Use volatile for: (1) simple flag (boolean stop), (2) publishing a reference to an immutable object, (3) double-checked locking (Java 5+). Use synchronized when you need atomicity for compound operations like check-then-act or read-modify-write.

Q: Why must wait() be called in a while loop, not an if?

A: Spurious wakeups: a thread can wake from wait() without notify() being called (JVM specification allows this). Using if checks the condition only once; using while re-checks it after every wakeup, ensuring the thread only proceeds when the condition is actually true.

Q: What is the difference between notify() and notifyAll()?

A: notify() wakes one arbitrarily chosen waiting thread. notifyAll() wakes all waiting threads -- each re-checks its while condition. Use notifyAll() to avoid liveness issues where the wrong thread is woken (one waiting for a condition that still doesn't hold).

Q: How do you implement cooperative cancellation in Java?

A: Set a volatile boolean stop flag or use Thread.interrupt(). The worker thread checks isInterrupted() in its loop, or blocking operations (sleep/wait/join) throw InterruptedException when the flag is set. Never use Thread.stop() -- it leaves shared state inconsistent.

Q: What should you do when you catch InterruptedException?

A: Either: (1) propagate it by declaring throws InterruptedException, OR (2) restore the interrupt flag with Thread.currentThread().interrupt() before returning. Never swallow it silently -- doing so hides the cancellation

signal from callers up the stack.

Q: Why is Executors.newFixedThreadPool() dangerous in production?

A: It uses an UNBOUNDED LinkedBlockingQueue. Under sustained overload, millions of tasks can queue up and exhaust heap memory (OutOfMemoryError). Always use ThreadPoolExecutor with a bounded queue (ArrayBlockingQueue) and a sensible RejectedExecutionHandler.

Q: What are the four RejectionExecutionHandler strategies?

A: AbortPolicy (default): throws RejectedExecutionException. CallerRunsPolicy: caller thread runs the task (provides back-pressure). DiscardPolicy: silently drops the task. DiscardOldestPolicy: drops the oldest queued task and retries submitting the new one.

Q: What are the four necessary conditions for deadlock?

A: 1) Mutual exclusion: resources held exclusively. 2) Hold and wait: holding one lock while waiting for another. 3) No preemption: locks cannot be forcibly taken. 4) Circular wait: A waits for B while B waits for A. Breaking any one prevents deadlock.

Q: How do you prevent deadlock with lock ordering?

A: Establish a global total order over all locks (e.g., by memory address or a fixed enum). Always acquire locks in that order, regardless of what the business logic requires. This prevents circular wait -- the fourth Coffman condition -- which is the most practical prevention strategy.

Q: What is a race condition? Give two examples.

A: An outcome that depends on the interleaving of threads. (1) Check-then-act: if(!map.containsKey(k)) map.put(k,v) -- two threads both see absent and both insert. (2) Read-modify-write: count++ -- two threads both read 0, both write 1, lost increment.

Q: What is the difference between AtomicInteger and volatile int?

A: volatile int provides visibility but not atomicity. count++ on volatile int is not atomic (3 ops: read, increment, write -- race condition). AtomicInteger.incrementAndGet() uses CAS (compare-and-swap) which is a single atomic hardware instruction -- no separate read-increment-write.

Q: What is happens-before in the JMM?

A: A JMM guarantee: if action A happens-before action B, the result of A is visible to B. Key rules: program order within a thread, monitor unlock before next lock, volatile write before next read, thread.start() before any action in the started thread.

Q: Why is double-checked locking unsafe without volatile?

A: Without volatile, the JVM may reorder constructor writes relative to the reference assignment: another thread can see instance != null but observe a partially initialized object. volatile prevents this reordering by enforcing happens-before between the write and read of instance.

Q: What is a virtual thread?

A: A lightweight JVM-managed thread (Java 21) that runs on carrier (platform) threads. When a virtual thread blocks on I/O, the JVM unmounts it from the carrier, freeing the carrier to run other virtual threads. Enables millions of concurrent threads with simple blocking code.

Q: [ADV] Explain the Java Memory Model happens-before for volatile.

A: A write to a volatile variable happens-before every subsequent read of that same variable. This means all memory actions performed before the volatile write are visible to the thread that performs the subsequent volatile read. This is why making a flag volatile ensures preceding writes (like x=42) are visible to the reader.

Q: [ADV] What is the monitor lock happens-before guarantee?

A: An unlock of a synchronized block/method happens-before any subsequent lock of the same monitor. This means every thread entering a synchronized block is guaranteed to see all writes made by all previous threads that held that same lock. This is the fundamental visibility guarantee of synchronized.

Q: [ADV] How does AtomicInteger.compareAndSet() work at the hardware level?

A: It maps to the CMPXCHG (compare-and-exchange) instruction on x86 or equivalent on other architectures. The CPU atomically: reads the memory location, compares with expected, if equal writes new value. This is a single indivisible hardware operation -- no lock needed. Hotspot uses sun.misc.Unsafe.compareAndSwapInt internally.

Q: [ADV] What is thread confinement and why does it avoid synchronization?

A: Confining an object to a single thread means only that thread can access it -- no sharing, no race conditions, no synchronization needed. Patterns: stack confinement (local variables), ThreadLocal, or explicit ownership transfer. JDBC Connection objects should be thread-confined (not shared between threads).

Q: [ADV] What is the happens-before for Thread.start() and Thread.join()?

A: start(): all actions in the calling thread before thread.start() happen-before any action in the started thread -- so the started thread sees all state set up by its creator. join(): all actions in the joined thread happen-before Thread.join() returns -- so the joining thread sees all final state written by the joined thread.

Q: [ADV] Explain LongAdder vs AtomicLong under high contention.

A: AtomicLong uses a single CAS location -- under high contention, many threads spin failing CAS and retry (thundering herd). LongAdder keeps an array of per-stripe cells and adds to a random cell. Reads (sum()) scan all cells. Under write-heavy workloads this is dramatically faster. Under low contention AtomicLong is faster (single cell).

Q: [ADV] What is false sharing and how do you prevent it?

A: False sharing: two independent variables on the same CPU cache line (typically 64 bytes). Thread A writes var1, Thread B writes var2 -- even though they are independent, each write invalidates the other's cache line, causing expensive cache coherence traffic. Fix: @Contended annotation (JVM adds padding) or manual padding with dummy fields.

Q: [ADV] What causes carrier thread pinning in virtual threads?

A: synchronized blocks and native methods pin the virtual thread to its carrier because the JVM cannot unmount it (the OS stack frame for synchronized monitor is on the carrier's stack). JNI (native) calls also prevent unmounting. Replace synchronized with ReentrantLock. Diagnose with -Djdk.tracePinnedThreads=full.

Q: [ADV] What is a ThreadLocal memory leak and how do you prevent it?

A: In a thread pool, threads are reused. If code sets a ThreadLocal value and never removes it, the value lives as long as the thread -- indefinitely in a pool. This holds references to objects that should have been GC'd. Fix: always call ThreadLocal.remove() in a finally block after use, or use ScopedValue (Java 21) which is automatically cleaned up.

Q: [ADV] Explain the sequential consistency vs relaxed memory model difference.

A: Sequential consistency: operations appear to execute in some global total order visible identically to all threads. Java does NOT guarantee sequential consistency -- it guarantees only happens-before ordering (a weaker model). Compilers and CPUs may reorder operations freely as long as happens-before edges are respected. This is why unsynchronized access to shared state can produce counterintuitive results.

12 Summary -- Multithreading Cheat Sheet

Topic	Key Rules	Common Mistakes
Thread	Call start() not run(); Runnable for tasks, Callable for values	Calling run() directly; starting same Thread twice
synchronized	Mutual exclusion + visibility; reentrant; use private lock	Locking on public object; static vs instance lock mismatch
volatile	Visibility only; no atomicity; use for flags and references	Using volatile for count++ (not atomic)
wait/notify	Must hold lock; while loop always; notifyAll over notify	Using if instead of while; calling without monitor lock
Interruption	Cooperative; propagate or restore flag; never swallow	Swallowing InterruptedException in empty catch block
Thread pools	Use bounded queue with ThreadPoolExecutor in production	Executors factory with unbounded queue causes OOM
Deadlock	Consistent lock ordering; tryLock with timeout	Acquiring locks in different orders in different threads
AtomicXxx	CAS for compound operations; LongAdder for high-contention counters	Using volatile for read-modify-write operations
JMM	Happens-before: program order, monitor, volatile, start, join	Assuming sequential consistency without synchronization
Virtual Threads	Replace synchronized with ReentrantLock; one thread per task	synchronized in virtual threads pins carrier thread